Libro de Prácticas Iker Núñez Meana
UO306425
Algoritmia

Tarea Practica 0 Iker Núñez UO306425

1) Factor 1:

Se pide: Reflejar en una tabla los tiempos de ejecución del programa PythonA1 para los valores de n desde 5000 hasta 80000. Si algún tiempo fuera superior a 60 segundos, se pone Fdt (Fuera de tiempo).

Ejercicio FACTOR1	Tiempos PythonA1				
n	**5000**	**10000**	**20000**	**40000**	**80000**
t (ms)	338	1352	5497	22838	Fdt

2) Factor 2:

Se pide: Ejecutar el algoritmo anterior en 2 máquinas diferentes, indicando RAM y procesador de cada ordenador.

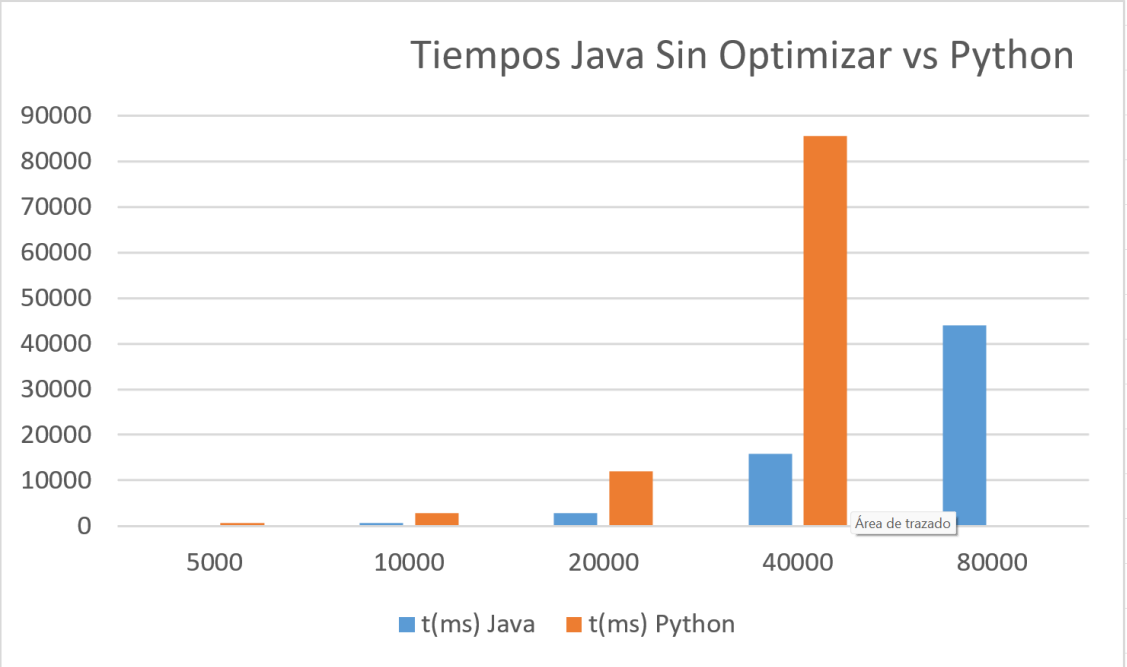
Ejercicio FACTOR2	Tiempos PythonA1				
12th Gen Intel(R) Core(TM) i5-12400 (2.50 GHz) clase					
RAM: 16,0 GB (15,8 GB usable) clase					
n	**5000**	**10000**	**20000**	**40000**	**80000**
t (ms)	338	1352	5497	22838	Fdt
13th Gen Intel(R) Core(TM) i7-13700H (2.40 GHz)					
16,0 GB (15,7 GB usable)					
n	**5000**	**10000**	**20000**	**40000**	**80000**
t (ms)	657	2835	12002	85574	Fdt

3) Factor 3:

Se pide: Implementar en Java el algoritmo usado en PythonA1.py, medir sus tiempos de ejecución con y sin optimización, para los mismos valores y compararlos con los de PythonA1.py.

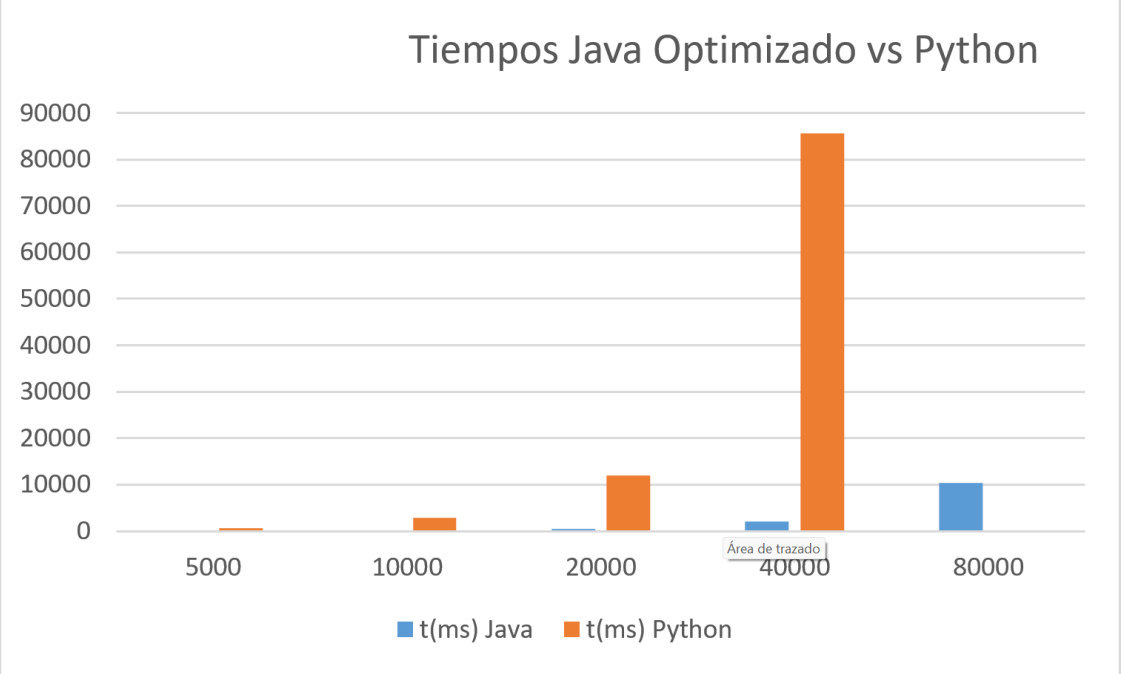
Tiempos para Java sin optimizar:

Python vs Java Sin Opt					
n	**5000**	**10000**	**20000**	**40000**	**80000**
t(ms) Java	179	668	2744	15881	43938
t(ms) Python	657	2835	12002	85574	Fdt



Tiempos para Java optimizado:

Python vs Java Con Opt					
n	5000	10000	20000	40000	80000
t(ms) Java	44	133	502	2025	10387
t(ms) Python	657	2835	12002	85574	Fdt



4) Factor 4:

Se pide: Hacer 3 programas en Java que usen el mismo algoritmo que PythonA2.py, PythonA3.py y PythonA4.py

Tras esto, medir los tiempos de ejecución para los 3 programas, con y sin

optimización, y representarlos en tablas frente a la ejecución de su análogo en Python para ver que entorno es más rápido.

Tiempos Algortimo A2

n	5000	10000	20000	40000	80000
t(ms) Java Sin Opt	48	62	210	769	2882
t(ms) Python	58	225	823	3126	11733
t(ms) Java Opt	27	14	42	159	594

Tiempos Algortimo A3

n	5000	10000	20000	40000	80000
t(ms) Java Sin Opt	29	42	146	524	1925
t(ms) Python	33	107	401	1514	5656
t(ms) Java Opt	28	9	22	89	302

Tiempos Algortimo A4

n	5000	10000	20000	40000	80000
t(ms) Java sin Opt	28	1	2	3	7
t(ms) Python	1	2	5	7	15
t(ms) Java Opt	27	1	3	2	2

En algunos casos, me salen tiempos inferiores para mayor n. Esto se debe a lo comentado en clase. La máquina virtual de Java guarda algunos resultados, luego al volver a pasar por esas operaciones tarda menos que la vez anterior.

Tarea Practica 1 Iker Núñez UO306425

1) Toma de tiempos de ejecución

Como currrentTimeMillis() devuelve un entero de 64 bits (long) que es el milisegundo en el que vive el ordenador actualmente. Luego, si quieres medir un tiempo de ejecución, no tienes que hacer más que llamar al método antes y después de ejecutarlo y restar después – antes.

La cuenta a cero se puso hace más de 50 años.

¿Cuántos años más podremos seguir utilizando este procedimiento para medir tiempos?

Bueno: como devuelve un entero de 64 bits, podremos representar 2^64 números, que son 18,446,744,073,709,551,616 números. Ahora, podemos ejecutar Vector2.java para algún n grande. Yo en mi caso escogí n = 100000000 y me salió t1 = 1770276323069.

Ahora podemos restar t1 – 2^64 y nos saldrá el número de años, pero le tendremos que restar 56 años, porque la cuenta a cero se puso hace 56 años.

Para no poner resultados diferentes a los obtenidos en clase, pongo el número de años que nos salieron en clase, pero no me acuerdo del n que utilizamos:

5.8 * 10^8 – 56 años.

¿Por qué a veces los tiempos de ejecución salen cero?

Esto puede ser por varias razones. Si estamos ejecutando Java con optimización, puede ser porque el compilador JIT se salta algunas instrucciones y recupera algunos resultados de ejecuciones anteriores, luego tarda menos.

En nuestro caso, estamos ejecutando Java sin optimización. La respuesta es que a veces, al ejecutar un programa en Java, el recolector de basura se ejecuta. Esto hace que se pausen todos los procesos, y por esta razón, los tiempos de ejecución salen cero.

¿A partir de que n se empiezan a obtener tiempos fiables?

Los tiempos de ejecución a veces pueden ser menores que 50 ms dependiendo del n que le pases. Estos tiempos serán descartados por falta de fiabilidad.

En el caso del programa Vector2.java, se empiezan a obtener tiempos fiables a partir de: 5.500.000 en mi caso desde el ordenador de casa, para el que se obtiene un tiempo de 51ms

```
PS C:\Users\IkerNuevo\Desktop\alg_Nu-ezMeanaIkerUO306425\p11> java -Xint p11.Vector2 100000000
t1=1770276323069  *** t2=1770276324017
n= 100000000    Tiempo metodo suma = 948
Resultado de la suma de elementos = 287119
PS C:\Users\IkerNuevo\Desktop\alg_Nu-ezMeanaIkerUO306425\p11> java -Xint p11.Vector2 1500000
t1=1770276850789  *** t2=1770276850803
n= 1500000      Tiempo metodo suma = 14
Resultado de la suma de elementos = 41180
PS C:\Users\IkerNuevo\Desktop\alg_Nu-ezMeanaIkerUO306425\p11> java -Xint p11.Vector2 1990000
t1=1770276869496  *** t2=1770276869513
n= 1990000      Tiempo metodo suma = 17
Resultado de la suma de elementos = -35769
PS C:\Users\IkerNuevo\Desktop\alg_Nu-ezMeanaIkerUO306425\p11> java -Xint p11.Vector2 10000000
t1=1770276879483  *** t2=1770276879578
n= 10000000     Tiempo metodo suma = 95
Resultado de la suma de elementos = -40015
PS C:\Users\IkerNuevo\Desktop\alg_Nu-ezMeanaIkerUO306425\p11> java -Xint p11.Vector2 7000000
t1=1770276903111  *** t2=1770276903176
n= 7000000      Tiempo metodo suma = 65
Resultado de la suma de elementos = -140129
PS C:\Users\IkerNuevo\Desktop\alg_Nu-ezMeanaIkerUO306425\p11> java -Xint p11.Vector2 5000000
t1=1770276912580  *** t2=1770276912627
n= 5000000      Tiempo metodo suma = 47
Resultado de la suma de elementos = 104830
PS C:\Users\IkerNuevo\Desktop\alg_Nu-ezMeanaIkerUO306425\p11> java -Xint p11.Vector2 5500000
t1=1770276920012  *** t2=1770276920063
n= 5500000      Tiempo metodo suma = 51
Resultado de la suma de elementos = 497
PS C:\Users\IkerNuevo\Desktop\alg_Nu-ezMeanaIkerUO306425\p11>
```

Adjunto una tabla con los tiempos para distintos n hecha en clase:

Vector 2					
n	100000	10000000	50000000	100000000	500000000
t(ms)	5	47	239	481	2412

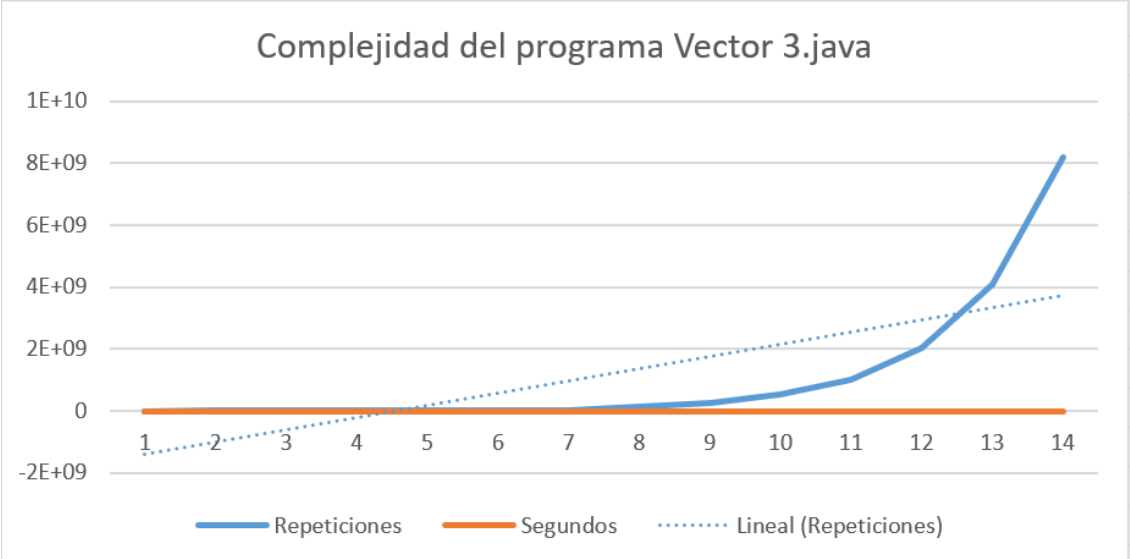
2) *Crecimiento del tamaño del problema*

A continuación, se adjunta una tabla con los tiempos de ejecución para distintos n SIN OPTIMIZACION como las ejecuciones anteriores:

Vector 3										
n	100000		10000000		50000000		100000000		500000000	
t(ms)	n	tTiempo	n	tTiempo	n	tTiempo	n	tTiempo	n	tTiempo
	10000	0	10000	0	10000	1	10000	0	10000	0
	20000	0	20000	0	20000	0	20000	0	20000	0
	40000	1	40000	0	40000	0	40000	0	40000	0
	80000	0	80000	0	80000	0	80000	1	80000	0
	160000	0	160000	1	160000	2	160000	2	160000	2
	320000	1	320000	2	320000	3	320000	2	320000	2
	640000	3	640000	4	640000	5	640000	6	640000	5
	1280000	6	1280000	6	1280000	18	1280000	12	1280000	11
	2560000	12	2560000	13	2560000	24	2560000	26	2560000	25
	5120000	24	5120000	27	5120000	47	5120000	63	5120000	56
	10240000	49	10240000	50	10240000	101	10240000	99	10240000	93
	20480000	98	20480000	99	20480000	210	20480000	182	20480000	190
	40960000	197	40960000	209	40960000	387	40960000	389	40960000	402
	81920000	393	81920000	408	81920000	771	81920000	748	81920000	763

Si cogemos los tiempos de una columna, los sacamos a una tabla y representamos la gráfica en una tabla, podemos ver la complejidad del algoritmo de forma gráfica. Realizándolo para n = 100000 por ejemplo:

Segundos	Repeticiones
0	10000
0	20000
1	40000
0	80000
0	160000
1	320000
3	640000
6	128000000
12	256000000
24	512000000
49	1024000000
98	2048000000
197	4096000000
393	8192000000

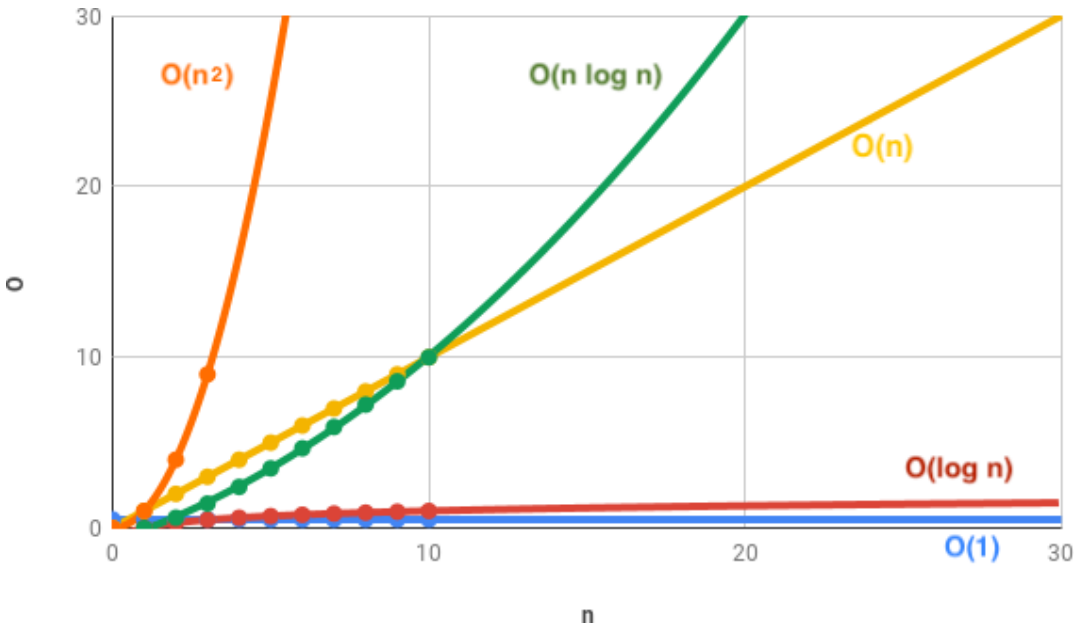


Como vemos, los tiempos que salen son solo los que se tardo en realizar el algoritmo suma, que tiene complejidad 0(n):

```
public static void main (String arg [] )
{
    long t1,t2;

    System.out.println (x: "n   tTiempo");
    for ( int n=10000; n<= 81920000; n*=2) // n se va incrementando (*2)
    {
        System.out.print (n+"\t");
        v = new int [n] ;
        Vector1.rellena (v);

        // Medida del tiempo de la operacion suma()
        t1=System.currentTimeMillis();
        int s=Vector1.suma (v);
        t2=System.currentTimeMillis();
        System.out.println ((t2-t1)+"\t");

        } // fin de for de n del problema

    System.out.println(x: "\nFin de la medicion de tiempos *****");
```

Luego, si nos fijamos en la línea de tendencia que sale en la gráfica anterior, vemos que la tendencia es lineal. Luego, los resultados pueden ser coherentes con la complejidad del algoritmo:

```
/* Este metodo suma los elementos de un vector y devuelve el resultado
*/
public static int suma (int[]a)
{
    int s=0;
    int n= a.length;
    for (int i=0;i<n;i++) s=s+a[i];
    return s;

}  // fin de suma
```



La inclinación en nuestra grafica es menos notable, pero es por los valores escogidos y la escala.

3) *Toma de tiempos pequeños*

Ejecuciones de Vector4.java:

1		10		100		1000		100000		10000000	
n	Tiempo	n	Tiempo	n	Tiempo	n	Tiempo	repeticiones = 100000		n	Tiempo
10000	0	10000	1	10000	14	10000	103	n	Tiempo	10000	89715
20000	0	20000	1	20000	17	20000	191	10000	6619	20000	202705
40000	1	40000	6	40000	38	40000	461	20000	13171	40000	661730
80000	1	80000	9	80000	73	80000	848	40000	26334	80000	1254631
160000	2	160000	18	160000	147	160000	1687	80000	53584	160000	2242497
320000	3	320000	28	320000	323	320000	3281	160000	106730	320000	4818522
640000	6	640000	72	640000	670	640000	6469	320000	210502	640000	10210934
1280000	12	1280000	127	1280000	1288	1280000	12958	640000	425768	1280000	25527904
2560000	27	2560000	283	2560000	4479	2560000	26837	1280000	835628	2560000	117180612
5120000	52	5120000	508	5120000	6985	5120000	52009	2560000	8733776		
10240000	159	10240000	1029	10240000	18683	10240000	101812	5120000	5401252		
20480000	271	20480000	1992	20480000	24603	20480000	269438	10240000	6841835		
40960000	462	40960000	3980	40960000	42466	40960000	348076	20480000	13674182		
81920000	1474	81920000	8163	81920000	142541	81920000	548334				

En mi caso, para n = 100000 y n = 1000000 corte la ejecución para los últimos números ya que tardaban más de 7h en hacer 2 iteraciones. Sin embargo, si miramos la tabla, podemos ver que para n = 100000 frente a n = 1000 se multiplican x50 e incluso por más de 60 para algún caso los tiempos:

13674182 / 269438 = 50,75

6841835 / 101812 = 67,20

Para n = 1000000 frente a n = 100000 pasa algo similar: 25527904 /

835628 = 30,55

Pero en menor proporción en este caso.

Las ejecuciones de este programa fueron cortadas al llegar a n altos debido al gran tiempo que tardaron en resolverse, el cual se puede ver en las capturas.

¿Qué pasaría si multiplicamos el tamaño del problema por 2?

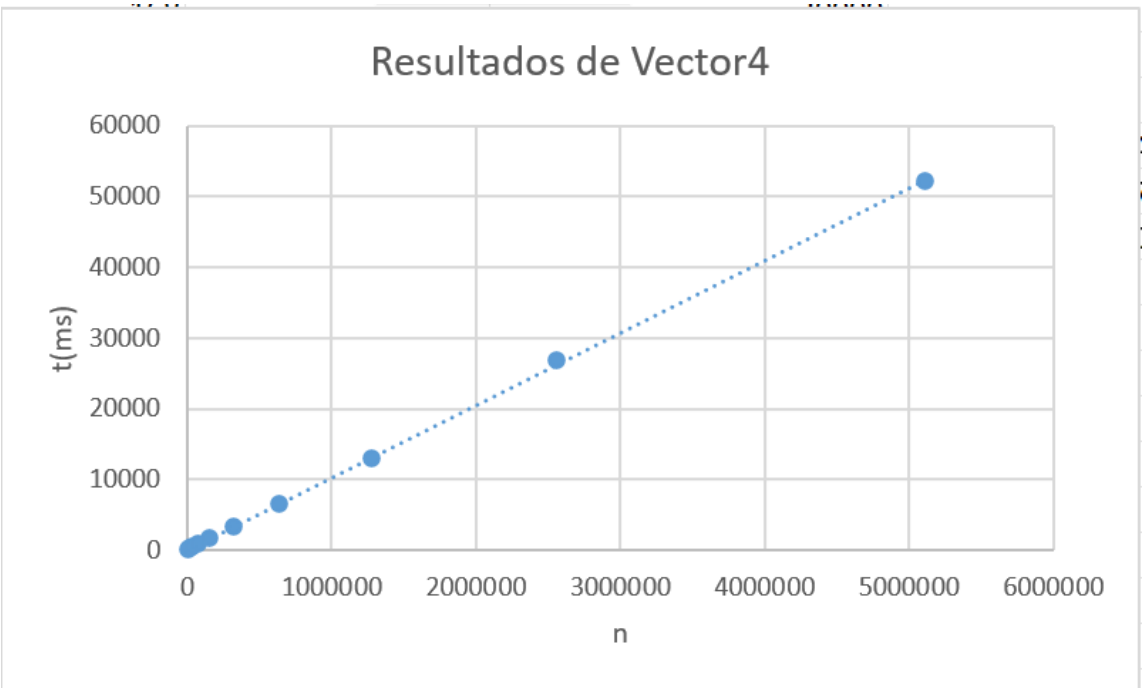
De acuerdo con la formula vista en la teoría, sabemos que si para un algoritmo de complejidad 0(lo que sea) y para n1 = x tarda un tiempo t1 = y segundos, entonces, si aumentamos el tamaño del problema en k, entonces t2 = k * t1.

¿Qué pasa si en vez de por 2 multiplicamos por otra constante (por ejemplo, probar con k = 3 y k = 4?

Pasaría lo mismo que el caso anterior donde k = 2, esta vez para k = 3 y k = 4.

¿Son los tiempos obtenidos razonables con la complejidad lineal del algoritmo?

Para comprobar esto, podemos representar los tiempos obtenidos en una gráfica y ver como sale la recta. En este caso, cojo, por ejemplo, los tiempos obtenidos para n = 1000:



Vemos que, para todos los puntos, sale la tendencia esperada por su complejidad (0(n)).

En la siguiente parte, se pide elaborar 3 programas más para medir los tiempos de otros algoritmos. Tras obtener los tiempos, se piden elaborar algunas tablas.

En todos los casos, pongo n = 1000 por ejemplo:

n	TiempoSuma	TiempoMaximo
10000	103	96
20000	191	168
40000	461	341
80000	848	682
160000	1687	1384
320000	3281	2712
640000	6469	5357
1280000	12958	10843
2560000	26837	21626
5120000	52009	43836
10240000	101812	88097
20480000	269438	18141411
40960000	348076	504529
81920000	548334	1600746

Para estas dos ejecuciones, use n = 10 ya que Vector6 tardaba mucho en ejecutarse:

n	TiempoCoincidencias1	TiempoCoincidencias2
10000	8359	2
20000	41484	4
40000	136952	5
80000	757538	12
160000	4411314	20
320000	17320862	39
640000	120958434	77
1280000	Fdt	164
2560000	fdt	339
5120000	fdt	660
10240000	fdt	1321
20480000	fdt	2708
40960000	fdt	5273
81920000	fdt	10829

¿Los resultados son coherentes con las complejidades de los algoritmos?

Primero, hemos de mirar cuales son las complejidades: Vector 4 ->

algoritmo de suma -> 0(n)

Vector 5 -> algoritmo de máximo -> 0(n) Vector 6 ->

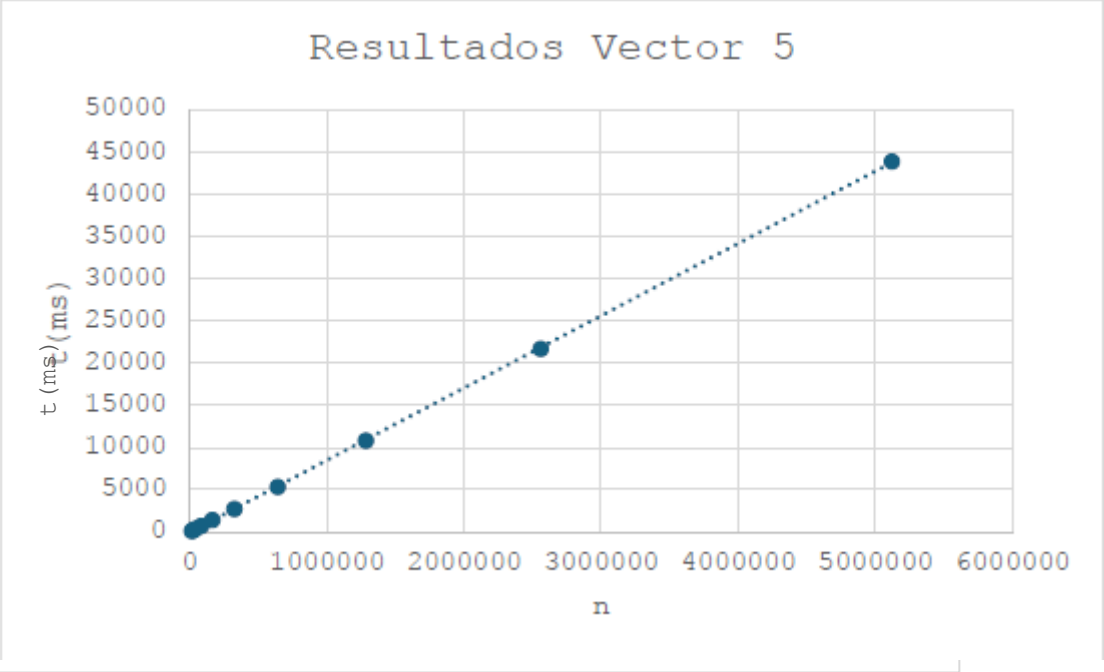
algoritmo coincidencias 1 -> 0(n^2) Vector 7 ->

algoritmo coincidencias 2 -> 0(n)

Los resultados del vector 4 ya vimos antes que, si eran coherentes, como cogí los mismos resultados no volveré sobre estos.

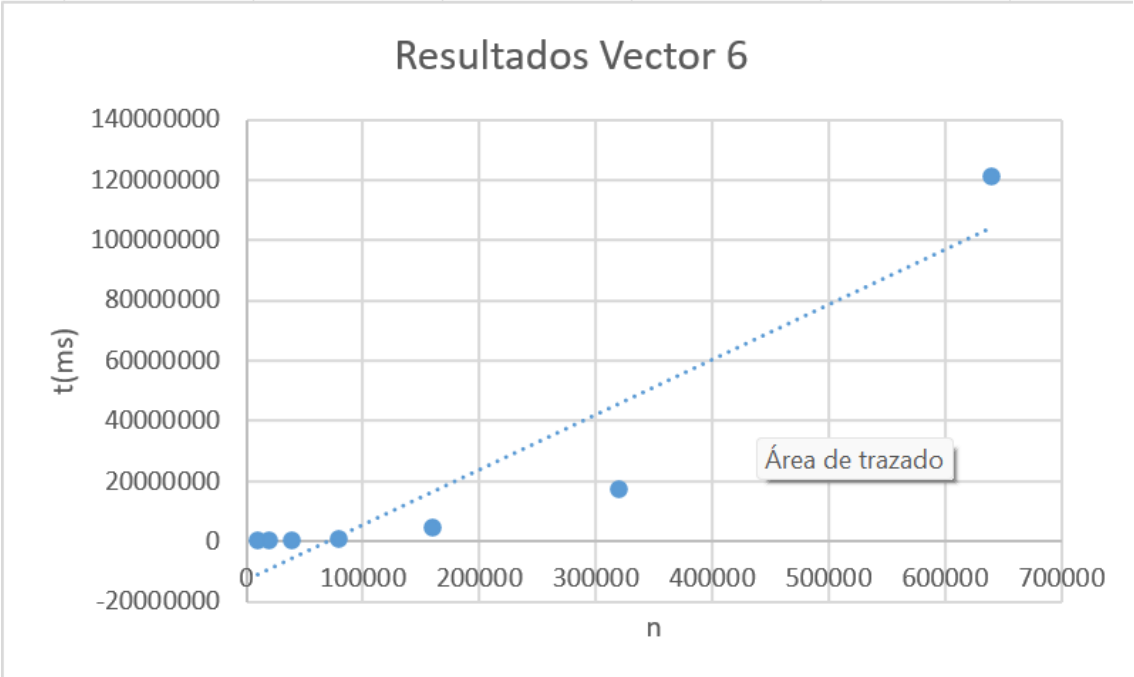
Para los resultados de Vector5:

En la siguiente gráfica, podemos ver los resultados visualmente:



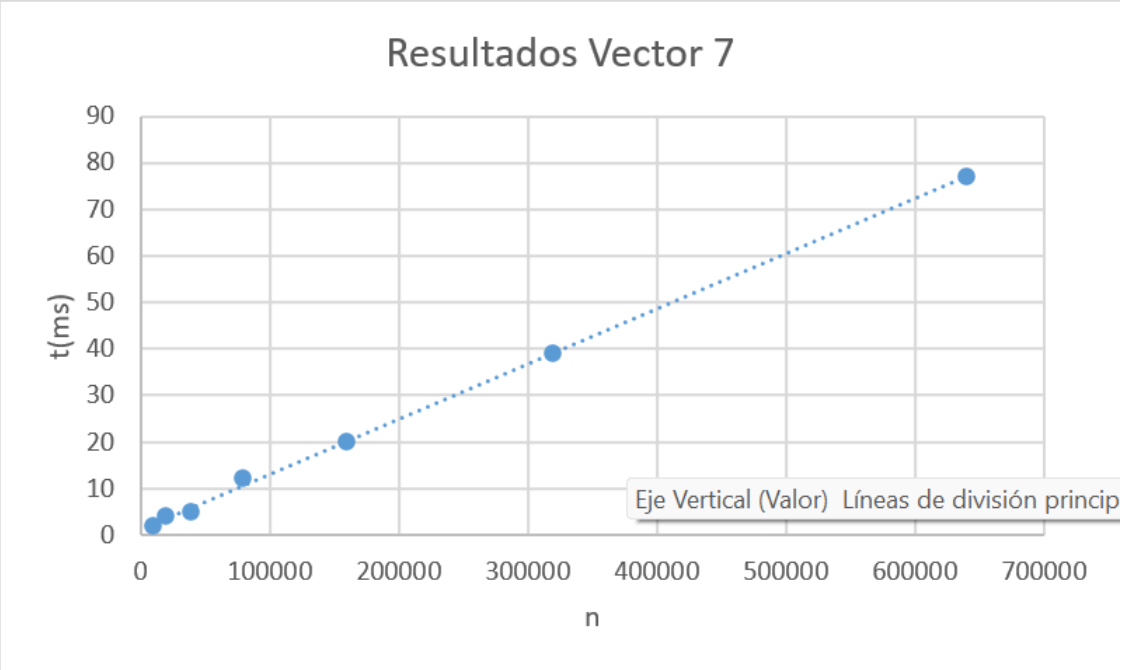
Si nos fijamos en la línea de tendencia, vemos que la tendencia es lineal, luego los resultados serían coherentes.

Para los resultados del Vector 6:



En este caso, vemos que la línea de tendencia tiene mayor inclinación que en la gráfica anterior. Sin embargo, al ser una complejidad cuadrática, quizás debería ser más inclinada aún. Por lo tanto, podemos decir que los resultados no son tan coherentes.

Para los resultados de Vector7:



Esta grafica certifica que los resultados de Vector6 no fueron los esperados, ya que la línea de tendencia tiene la misma inclinación y la complejidad de este algoritmo es 0(n), por tanto, para Vector7, los resultados si pueden ser coherentes, pero para Vector6 no.

Las características del ordenador donde he medido los tiempos son: Apartados 1 y 2:

Ordenador de clase

Procesador: 12th Gen Intel® Core™ i5-12400 (2.50GHz) Memoria

RAM: 16,0 GB (15,8 GB usable)

Resto de las mediciones:
Ordenador personal
Procesador: 13th Gen Intel(R) Core(TM) i7-13700H (2.40 GHz) Memoria: 16,0 GB (15,7 GB usable)

Tarea Práctica 2 Iker Núñez UO306425

1) Complejidades temporales de los algoritmos

Bucle 1-> 0(n * log n)
Bucle 2-> 0(n^2 * log n)
Bucle 3 -> 0(n^2 * log n)
Bucle 4 -> 0(n^3)

Luego, tendremos:
Bucle 5-> 0(n^2 * log^2(n))
Bucle 6 -> 0(n^3 * log n)
Bucle 7 -> 0(n^4)

2) Algunos modelos iterativos

A continuación, se pide medir los tiempos de los 4 primeros algoritmos. Para que el Bucle4 no se saliera fuera de tiempo, utilizamos un parámetro de repeticiones = 1, y aun, así como veremos las ultimas 3 n se salieron fuera de tiempo:

n	Bucle1	Bucle2	Bucle3	Bucle4
100	0	1	1	1
200	0	0	4	6
400	0	3	15	44
800	0	18	66	331
1600	0	61	262	2805
3200	1	242	1105	22250
6400	1	1092	4729	175379
12800	3	4608	19979	fdt
25600	6	18927	83250	fdt
51200	10	76024	343020	fdt

3) Creación de modelos iterativos de una complejidad dada

Se piden implementar otras 3 clases: Bucle5, Bucle6 y Bucle7 en Java, modificando las anteriores de manera que tengan la complejidad pedida arriba. Tras implementarlas, volvemos a medir los tiempos para repeticiones = 1:

n	Bucle5	Bucle6	Bucle7
100	1	10	45
200	4	91	582
400	18	805	15305
800	91	7020	265501
1600	444	137974	10148737
3200	2131	1207614	fdt
6400	10019	fdt	fdt
12800	46971	fdt	fdt
25600	211785	fdt	fdt
51200	fdt	fdt	fdt

Los resultados son los esperados, el tiempo va aumentando en los 3 bucles debido a que aumenta la complejidad.

4) *Relación temporal de algoritmos*

Caso 1: Algoritmos de distinta complejidad:

En este caso, como vamos a poner el de menor complejidad arriba, a medida que n crezca, el cociente debería aproximarse a 0, ya que denominador > numerador:

n	Bucle1	Bucle2	t1/t2
100	0	1	0
200	0	0	0
400	0	3	0
800	0	18	0
1600	0	61	0
3200	1	242	0,00413223
6400	1	1092	0,00091575
12800	3	4608	0,00065104
25600	6	18927	0,00031701
51200	10	76024	0,00013154

Efectivamente, los resultados son los esperados, tanto por la tendencia del cociente, como que los tiempos de bucle 1 son inferiores (bastante) a los de bucle 2.

Caso 2: Algoritmos de misma complejidad:

Medir tiempos es interesante por esto. A pesar de que dos algoritmos tengan la misma complejidad, pequeños detalles en la implementación pueden hacer que uno tarde mucho más que otro. Por esta razón, es importante no solo mirar la complejidad, sino mirar también la estructura del código.
En este caso, vamos a poner el que más tarda en el numerador, luego como numerador > denominador, el cociente debería tender a infinito (o en nuestro caso para pocas n crecer sin más):

n	Bucle3	Bucle2	t3/t2
100	1	1	1
200	4	0	4
400	15	3	5
800	66	18	3,66666667
1600	262	61	4,29508197
3200	1105	242	4,5661157
6400	4729	1092	4,33058608
12800	19979	4608	4,33572049
25600	83250	18927	4,39847836
51200	343020	76024	4,51199621

Efectivamente, los resultados son los esperados, tanto por el cociente, como por los tiempos de ambos algoritmos (t3 >> t2)

Caso 3: Mismo algoritmo en diferentes entornos de desarrollo:
Para terminar, vamos a comparar 2 entornos de ejecución: CPython y Java para ver cuál es más rápido ejecutando exactamente el mismo algoritmo.
Java debería ser más rápido, simplemente por ser compilado y no interpretado. Pero, además, en cada iteración se incrementa una variable contador que es un entero. Sin embargo, como Python no tiene tipos, deberá comprobar en cada iteración, de que tipo es la variable. Java no. Solo ahí habría una diferencia de tiempos.

n	Bucle4 Py	Bucle4 Java	tPython/tJava
100	6	1	6
200	50	6	8,33333333
400	321	44	7,29545455
800	2750	331	8,3081571
1600	31326	2805	11,1679144
3200	449582	22250	20,2059326
6400	fdt	175379	5,7019E-06

Los resultados son los esperados, pues java es más rápido. El cociente también es razonable, pues va creciendo.

Tarea Practica P2 Iker Núñez UO306425

1) Tabla 1: Algoritmo de ordenación Burbuja

Para n = 1 en los tres casos y sin optimización:

n	Ordenado	Inverso	Aleatorio
10000	418	1096	851
20000	1733	4372	3515
40000	6924	17544	14080
80000	27423	69033	55686
160000	109910	276233	222360
320000	fdt	fdt	fdt
640000	fdt	fdt	fdt
1280000	fdt	fdt	fdt

¿Son los tiempos obtenidos los esperados?

Para razonarlo, nos apoyamos en la implementación:

```
/* Ordenacion por el metodo de Burbuja */
public static void burbuja(int[] a) {
    int n = a.length;
    int x;
    for (int i = 0; i <= n - 2; i++)
        for (int j = n - 1; j > i; j--)
            if (a[j - 1] > a[j]) {
                // intercambio de valores
                x = a[j - 1];
                a[j - 1] = a[j];
                a[j] = x;
            }
}
```

En este algoritmo, el array de entrada no es muy notable en el tiempo que tarde, porque en todos los casos recorre el array entero por pares, por los dos for's anidados, luego la diferencia estará en todos los casos en el número de veces que entra por el if / no.

En el caso del array ordenado, no entrara nunca por el if por eso es el más rápido de todos, sin embargo, no es mucho más rápido por lo comentado. Si el array de entrada esta ordenado de forma aleatoria, algunas veces entrará por el if y otras no, por eso tarda más que el ordenado.

Para el último caso, un array ordenado inversamente, entrará por el if en todos los casos, por eso es el más lento, pero no por mucho.

2) *Tabla 2: Algoritmo de ordenación Selección*

Para n = 1 en los tres casos y sin optimización:

n	Ordenado	Inverso	Aleatorio
10000	395	349	396
20000	1614	1380	1579
40000	6372	5625	6337
80000	25329	22125	25334
160000	100812	88357	100603
320000	fdt	fdt	fdt
640000	fdt	fdt	fdt
1280000	fdt	fdt	fdt

¿Son los tiempos obtenidos los esperados?

Para razonarlo, nos apoyamos en la implementación:

```
/* Ordenacion por el metodo de Seleccion */
public static void seleccion(int[] a) {
    int n = a.length;
    int x;
    int posmin;
    for (int i = 0; i < n - 1; i++) { // Buscar la posicion del mas
    // pequeno de los que quedan
        posmin = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[posmin])
                posmin = j;
        // Intercambia el que toca con el mas peque►o
        x = a[posmin];
        a[posmin] = a[i];
        a[i] = x;

    } // for
}
```

En este caso, lo que hace es para cada i busca el número más bajo desde i + 1 hasta el final y, si este número es más bajo que el que está en la posición i, intercambia su posición este.

Luego, para un array ordenado, recorrerá todo el array n*n veces por los dos for's anidados y no hará modificación alguna ya que nunca encontrará un número más bajo por el que intercambiar la posición.

Para un array aleatorio, la diferencia en tiempo es poco notable, ya que algunas veces hará modificaciones y otras no, pero recorrerá el array el mismo número de veces.

Por último, para un array ordenado inversamente, se dan los mejores tiempos ya que hará modificaciones constantes pero rápidas e ira ordenando el array. Es por esto que los tiempos en este caso son los más bajos.

3) *Tabla 3: Algoritmo de ordenación Inserción*

Para n = 1 en los tres casos y sin optimización:

n	Ordenado	Inverso	Aleatorio
10000	0	683	338
20000	1	2698	1359
40000	1	10774	5382
80000	2	43098	21628
160000	3	172258	86641
320000	6	fdt	356737
640000	12	fdt	fdt
1280000	24	fdt	fdt

¿Son los tiempos obtenidos los esperados?

Para razonarlo, nos apoyamos en la implementación:

```
/* Ordenacion por el metodo de Insercion */
public static void insercion(int[] a) {
    int n = a.length;
    for (int i = 1; i <= n - 1; i++) {
        int x = a[i];
        int j = i - 1;
        while (j >= 0 && x < a[j]) {
            a[j + 1] = a[j];
            j = j - 1;
        }
        a[j + 1] = x;
    } // for
}
```

Lo que hace el algoritmo de inserción es iterar desde el primer elemento y crear 2 particiones, una que será la ordenada (a su izquierda) y otra que es la que está sin ordenar (a su derecha).

Para un array de entrada ordenado realmente no hace nada, solo recorre el array pero el resto son operaciones O(1) ya que en el while no entra nunca al no ser x< a[j] (es decir x menor que el anterior). Luego es por eso que los tiempos salen tan pequeños.

Para el caso de un array aleatorio, algunas veces entrara en el while y otras no, por eso tarda más que en el caso del array ordenado.

Por último, el array ordenado inversamente es el peor caso en cuanto a tiempos, ya que en todos los casos va a entrar en el while y de hecho va a tener que ir moviendo todas las posiciones a la izquierda por el hecho de que este ordenado inversamente. Por ello, es el peor caso en cuanto a tiempos con diferencia.

4) *Tabla 4: Algoritmo de ordenación Rápido*

Para n = 1 en los tres casos y sin optimización:

n	Ordenado	Inverso	Aleatorio
10000	4	4	4
20000	6	7	9
40000	12	13	17
80000	24	26	35
160000	51	61	75
320000	112	129	169
640000	240	254	343
1280000	493	528	710

¿Son los tiempos obtenidos los esperados?

Para razonarlo, nos apoyamos en la implementación:

```
public static void rapido(int[] v) {
    rapirec(v, iz: 0, v.length - 1);
}

private static void rapirec(int[] v, int iz, int de) {
    int m;
    if (de > iz) {
        m = particion(v, iz, de);
        rapirec(v, iz, m - 1);
        rapirec(v, m + 1, de);
    }
}
```

```java
private static void intercambiar(int[] v, int i, int j) {
    int t;
    t = v[i];
    v[i] = v[j];
    v[j] = t;
}

/**
 * Deja el pivote en una posicion tal que a su izquierda no
 * hay ningun mayor, ni a la derecha ningun menor.
 * Es un proceso lineal O(n).
 */

private static int particion(int[] v, int iz, int de) {
    int i, pivote;
    intercambiar(v, (iz + de) / 2, iz);
    // el pivote es el de centro y se cambia con el primero
    pivote = v[iz];
    i = iz;
    for (int s = iz + 1; s <= de; s++)
        if (v[s] <= pivote) {
            i++;
            intercambiar(v, i, s);
        }
    intercambiar(v, iz, i);// se restituye el pivote donde debe estar
    return i; // retorna la posicion en que queda el pivote
}
```

En este caso, nos encontramos con el algoritmo de QuickSort, que es una implementación de un algoritmo de tipo Divide y Vencerás. En este caso, se escoge un pivote que es el primer elemento de la partición y lo que se hace es ir haciendo el problema mas pequeño hasta que resulte una operación simple.

En este caso, vemos que el array de entrada no afecta demasiado a los tiempos de ejecución, este ordenado o no. Esto se debe a que simplemente no mira cada par de elementos, sino solo compara con el pivote escogido tras hacer las particiones. Los tiempos salen relativamente parecidos en todos los casos, pero vemos que tiene sentido que lo sean.

A partir de las complejidades y de los datos anteriores, ¿cuántos días tardaría cada algoritmo en ordenar un vector de 81.920.000 para un array ordenado de forma aleatoria?

Algoritmo de Burbuja:

Datos: n1 = 160000, t1 = 222360 ms n2 =

81.920.000 , t2 = ¿?

Complejidad del algoritmo: O(n^2) en todos los casos (pdf teoría) t2 = 222360 *

((81.920.000)^2 / (160000)^2) = 5,82903 * 10^10 ms 1 día -> 86.400.000 ms

Luego,
5,82903 * 10^10 * 1 día / 86.400.000 = 674,66 días -> 675 días

Algoritmo de Inserción:

Datos: n1 = 320000, t1 = 356737 ms n2 =

81.920.000 , t2 = ¿?

Complejidad del algoritmo: O(n^2) en este caso, que no es Caso Mejor (pdf teoría)

t2 = 356737 * ((81.920.000)^2 / (320000)^2) = 2,33791 * 10^10 ms 1 día ->

86.400.000 ms

Luego,

2,33791 * 10^10ms * 1 día / 86.400.000ms = 279,59 días -> 280 días

Algoritmo de Selección:

Datos: n1 = 160000, t1 = 100603 ms n2 =

81.920.000 , t2 = ¿?

Complejidad del algoritmo: O(n^2) en todos los casos (pdf teoría) t2 = 100603 *

((81.920.000)^2 / (160000)^2) = 2,63724 * 10^10 ms 1 día -> 86.400.000 ms

Luego,

2,63724 * 10^10ms * 1 día / 86.400.000ms = 305,24 días -> 306 días

Algoritmo de QuickSorting:

Datos: n1 = 1280000, t1 = 710ms n2 =

81.920.000 , t2 = ¿?

Complejidad del algoritmo: O(n*log(n)), tiramos por el caso medio.

t2 = 710 * ((81920000 * log(81920000)) / (1280000 * log(1280000))) = 58878,76732 ms

1 día -> 86.400.000 ms

Luego,
58878,76732ms * 1 día / 86.400.000ms = 6,81466*10^-4 días -> Menos de un día

5) Tabla 5: Rápido vs Inserción en tiempos bajos

Para n = 100000, sin optimización y orden aleatorio en ambos casos:

n	Rapido	Insercion	t rapido/t insercion
10	230	126	1,825396825
15	392	226	1,734513274
20	582	335	1,737313433
25	726	498	1,457831325
30	875	845	1,035502959
35	1068	1570	0,680254777
40	1589	1271	1,250196696
45	1900	2225	0,853932584
50	2301	2621	0,877909195
55	3055	2675	1,142056075
60	3487	3603	0,967804607
65	3179	3775	0,842119205
70	4358	5084	0,857199056
75	4960	5428	0,913780398
80	5289	5892	0,897657841
85	5599	6955	0,805032351
90	6195	7683	0,806325654
95	6265	6155	1,017871649
100	5802	6629	0,875245135

El algoritmo de QuickSort empieza a ser más rápido que el de inserción a partir de n = 35, para n's más bajos el algoritmo de inserción es más rápido. Esto nos quiere decir que, a pesar de que el algoritmo de Quicksort sea el mejor algoritmo de ordenación de todos por complejidad, para tamaños pequeños no es significante el algoritmo que uses en cuanto a tiempos de ejecución.

De forma opcional, se mandaba hacer un programa que según el tamaño del array eligiera el método de inserción o el método de Quicksort. En mi caso, Inserción era más rápido para arrays de tamaño <= 30. El programa principal sería este:

```
Run | Debug
public static void main(String arg[]) {
    long t1, t2;
    String opcion = arg[0];
    int repeticiones = Integer.parseInt(arg[1]);

    for (int n = 10; n <= 100; n += 5) {
        v = new int[n];
        long t = 0;

        for (int i = 1; i <= repeticiones; i++) {
            Vector.ordenAleatorio(v);

            t1 = System.currentTimeMillis();

            RapidoConInsercion.ordenar(v);

            t2 = System.currentTimeMillis();
            t = t + (t2 - t1);
        }

        System.out.println("n=" + n + "\tTiempo=" + t + "  \tRepeticiones=" + repeticiones);
    }
```

Y la implementación del método ordenar sería la siguiente:

```
/* Ordenacion segun el tamaño del array de entrada */
public static void ordenar(int [] a){
    int n = a.length;

    //hasta n = 30, el algoritmo de insercion es mas rapido

    if(n <= 30){
        insercion(a);
    } else{
        rapido(a);
    }
}

/* Ordenacion por el metodo de Insercion */
private static void insercion(int[] a) {
    int n = a.length;
    for (int i = 1; i <= n - 1; i++) {
        int x = a[i];
        int j = i - 1;
        while (j >= 0 && x < a[j]) {
            a[j + 1] = a[j];
            j = j - 1;
        }
        a[j + 1] = x;
    } // for
}

private static void rapido(int[] v) {
    rapirec(v, iz: 0, v.length - 1);
}
```

Si el array de entrada tiene un tamaño menor o igual que 30 llama al método de ordenar por Inserción, sino al de Quicksort.

Los tiempos resultantes para un orden aleatorio como el pedido y n = 100000 repeticiones (porque así podemos compararlos con los de la tabla 5 anterior):

n	t(ms)
10	109
15	218
20	350
25	556
30	1165
35	1798
40	2131
45	2606
50	2815
55	3258
60	3522
65	3908
70	4231
75	4618
80	5087
85	5553
90	5851
95	6000
100	6528

n	Rapido	Insercion	t rapido/t insercion
10	230	126	1,825396825
15	392	226	1,734513274
20	582	335	1,737313433
25	726	498	1,457831325
30	875	845	1,035502959
35	1068	1570	0,680254777
40	1589	1271	1,250196696
45	1900	2225	0,853932584
50	2301	2621	0,877909195
55	3055	2675	1,142056075
60	3487	3603	0,967804607
65	3179	3775	0,842119205
70	4358	5084	0,857199056
75	4960	5428	0,913780398
80	5289	5892	0,897657841
85	5599	6955	0,805032351
90	6195	7683	0,806325654
95	6265	6155	1,017871649
100	5802	6629	0,875245135

Como vemos, para n's pequeños, efectivamente, los tiempos son mejores que en Quicksort y a partir de 30 los tiempos son mejores que en la otra tabla con Inserción. Luego, el algoritmo está funcionando bien.

Tarea Sesión 5 Iker Núñez UO306425

1) DyV por Sustracción

Complejidades de los siguientes programas:

Complejidad Sustraccion1 -> O(n) porque a = 1 (1 llamada recursiva), b = 1 (hace n – 1 en cada llamada) y k = 0 (porque la complejidad sin contar la llamada recursiva es O(1)). Por tanto, siguiendo la fórmula para a = 1, O(n^k+1) □ O(n)

Complejidad Sustraccion2 -> en este caso, a = 1 (1 llamada recursiva), b = 1 (hace n – 1) y k = 1 (porque sin recursividad es O(n). Por tanto, siguiendo la fórmula para a = 1, O(n^k+1) □ O(n^2)

Complejidad Sustraccion3 -> en este caso, a = 2 (2 llamadas recursivas), b = 1 (hace n – 1 en cada llamada) y k = 0 (porque sin recursividad es O(1)), luego siguiendo la fórmula para a > 2 □ O(2^n/1) = O(2^n)

Los tiempos coinciden porque Sustraccion1 y Sustraccion2 rompen muy rápido porque cogen n muy grandes, pero porque la pila de Java tiene un tamaño limitado, entonces cuando hay tantas llamadas recursivas, rompe. Con Sustraccion3, pasa lo mismo de forma mas agresiva, ya que la complejidad es exponencial en este caso, pero como coge n's muy pequeños tarda más en cascar.

¿Para qué n abortan ejecución Sustraccion1 y Sustraccion2?

Sustraccion1 -> n = 8000
Sustraccion2 -> n = 8000
Ocurre por lo comentado antes, la pila de java se llena y da StackOverflow.

¿Cuántos años tarda en finalizar Sustraccion3 para n = 80?

La complejidad de Sustraccion3 es O(2^n). Luego,
n1 = 32, t1 = 177744ms, n2 = 80 -> t2 = (2^80/2^32) * 177744 ms = 5.0030 * 10^19 ms.
Para saber el número de años, habría que aplicar el siguiente factor de conversión:
5.0030 * 10^19 * 1 sec / 1000 ms * 1 hora / 3600 sec * 1 dia/24 horas + 365 = **2.1135 * 10^14 años**.

Sustracción 4 -> Implementación y tiempos:

```
public static void rec2 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        for (int i=0;i<n;i++) cont++; // O(n)
        rec2 (n-1);
        for (int i=0;i<n;i++) cont++; // O(n)
    }
}
```

n	t(ms)
1000	12
2000	58
4000	215
8000	862
16000	fdt

Sustracción 5 -> Implementación y tiempos:

```
public static void rec2 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        rec2 (n-2);
        rec2 (n-2);
        rec2 (n-2);
    }
}
```

n	t(ms)
10	0
20	4
40	267122
80	fdt

Lo pare porque al tener una complejidad tan mala, con n's muy pequeños ya tarda mucho.

¿Cuánto tardaría en finalizar la ejecución para n = 80 para Sustraccion5?

Regla de 3 para n1, t1, y n2 = 80, usando la complejidad de Sustraccion5 -> O(3^n/2)

DyV por División

Complejidades de los siguientes programas:
Division1 -> O(n) porque a = 1, b = 3 y k = 1 -> a < b^k □ O(n^1) = O(n)
Division2 -> O(n * log n) porque a = 2, b = 2 y k = 1 -> a = b^k □ O(n * log n)
Division3 -> porque a = 2, b = 2 y k = 0 -> a > b ^ k □ O(n^log2(2)) Division4 ->

Implementación y tiempos:

```
public static void rec4 (int n)
    {
        if (n<=0)
            cont++;
        else
        {
            cont++ ; // O(1)
            for(int i = 0; i < n; i++){
                for(int j = 0; j < n; j++){

                }
            }
            rec4 (n/2);
        }

    }
```

n	t(ms)
1000	11
2000	32
4000	126
8000	504
16000	2044
32000	9885
64000	50540
120000	176178

División5 -> Implementación y tiempos:

```
public static void rec5 (int n)
    {
        if (n<=0)
            cont++;
        else
        {
            cont++ ; // O(1)
            rec5 (n/2);
            rec5 (n/2);
            rec5 (n/2);
            rec5 (n/2);
        }

    }
```

n	t(ms)
1000	46
2000	183
4000	833
8000	3283
16000	22769
32000	80135
64000	306691

1) *Dos ejemplos básicos*

Complejidades de los siguientes programas:
VectorSuma:
Sum1 -> O(n)

Sum2 -> a = 1 (hay una llamada recursiva), b = 1 (porque hace i + 1) y k = 0 (porque fuera de la recursividad es O(1)). Luego, como a = b^k, entonces O(n^k * log n), pero como k = 0 □ O(log n)

Sum3 -> a = 2 (hay una llamada recursiva), b = 2 (divide entre 2) y k = 0 (fuera de la recursividad es O(1)). Luego, como a > b^k, entonces O(n^(log2 2)).

Tiempos de VectorSuma:

Suma	Sum1	Sum2	Sum3
532	680	225	4155

Fibonacci:

Fib1 -> O(n) Fib2 ->
O(n)
Fib3 -> a = 1 (1 llamada recursiva), b = 1 (porque va haciendo n – 1 en la llamada) y k = 0 (no hay mas que operaciones O(1) fuera). Por tanto, como es por sustracción -> a = 1 □O(n^k+1) = O(n)

Fib4 -> por sustracción. Hay dos llamadas recursivas -> a = 2, b no lo podríamos determinar de forma fija ya que en un caso es 1 y en otro 2, y k = 0 ya que fuera solo hay operaciones O(1). Luego, a > 1 □ O(a^n/b) = O(2^n/b) donde b variable. Podríamos decir que la complejidad esta entre: O(2^n) y O(2^n/2)

Tiempos de Fibonacci:

n	Suma	Fib1	Fib2	Fib3	Fib4
40	102334155	329	472	891	5957

2) *Practica DyV*

Empezamos implementando el algoritmo trivial que busca la distancia mínima. Probamos con los diferentes ficheros y vemos que sale correcto.

A continuación, se presenta la implementación:

```java
public class PuntosTrivial
{
    static double[][] matrizPuntos;
    static int tamañoMatriz;
    static ArrayList<String> lineas = new ArrayList<String>();
    static Random rand = new Random();

    /*
     * Metodo para leer de fichero
     * @param nombreFichero
     */
    public static void leerFichero(String nombreFichero){
        BufferedReader bf = null;
        try
        {
            bf = new BufferedReader(new FileReader(nombreFichero));
            String linea;
            boolean isFirstLine = true;
            int indice = 0;

            while((linea = bf.readLine()) != null)
            {
                if(isFirstLine)
                {
                    tamañoMatriz = Integer.parseInt(linea);
                    matrizPuntos = new double[tamañoMatriz][2];
                    isFirstLine = false;
                } else
                {
                    String[] lineaDiv = linea.split(regex: ",S");
                    matrizPuntos[indice][0] =
                        Double.parseDouble(lineaDiv[0]);
                    matrizPuntos[indice][1] =
                        Double.parseDouble(lineaDiv[1]);
                    indice++;
                }

            }
```

```java
public class PuntosTrivial
    public static void leerFichero(String nombreFichero){
        } catch (IOException e)
        {
            e.printStackTrace();
        } finally
        {
            try{
                if(bf != null)
                {
                    bf.close();
                }
            } catch(IOException e)
            {
                e.printStackTrace();
            }
        }
    }

    public static void mezclar()
    {
        for(int i = 0; i < matrizPuntos.length; i++)
        {
            int j = rand.nextInt(i + 1);

            double[] temporal = matrizPuntos[i];
            matrizPuntos[i] = matrizPuntos[j];
            matrizPuntos[j] = temporal;
        }
    }

    public static String[] BuscarDistanciaMinima()
    {
        double minDistancia = 100000000;
        int indice1 = -1;
        int indice2 = -1;
        String[] resultado = new String[2];

        for(int i = 0; i < matrizPuntos.length; i++)
        {
```

```java
public class PuntosTrivial

    public static String[] BuscarDistanciaMinima()
    {
        double minDistancia = 100000000;
        int indice1 = -1;
        int indice2 = -1;
        String[] resultado = new String[2];

        for(int i = 0; i < matrizPuntos.length; i++)
        {
            for(int j = i + 1; j < matrizPuntos.length - 1; j++)
            {
                double x1 = matrizPuntos[i][0];
                double y1 = matrizPuntos[i][1];

                double x2 = matrizPuntos[j][0];
                double y2 = matrizPuntos[j][1];

                //formula de la distancia entre 2 puntos: Raiz((x2-x1)^2 + (y2-y1)^2)
                double distancia = Math.sqrt((Math.pow((x2 - x1), b: 2)) + Math.pow((y2 - y1), b: 2));

                if(distancia < minDistancia){
                    minDistancia = distancia;
                    indice1 = i;
                    indice2 = j;
                }
            }
        }

        resultado[0] = String.format(format: "SU DISTANCIA MINIMA: %.6f", minDistancia);
        resultado[1] = String.format(format: "PUNTOS MAS CERCANOS: [%.6f,%.6f] [%.6f,%.6f]",
            matrizPuntos[indice1][0], matrizPuntos[indice1][1],
            matrizPuntos[indice2][0], matrizPuntos[indice2][1]);

        return resultado;
    }
```

La complejidad es claramente O(n^2) ya que tenemos dos bucles for anidados.

Y una clase PuntosTrivialTiempos que repite el algoritmo i veces y mide los tiempos de ejecución:

```
import p3p.PuntosTrivial;

public class PuntosTrivialTiempos
{
    Run | Debug
    public static void main(String[] args)
    {
        String nombreFichero = args[0];

        long t1,t2;
        String[] resultado;
        PuntosTrivial.leerFichero(nombreFichero);

        for (int i = 1024; i <= 1000000; i *= 2)
        {
            //Ordenamos las filas de forma aleatoria
            PuntosTrivial.mezclar();

            t1 = System.currentTimeMillis ();

            resultado = PuntosTrivial.BuscarDistanciaMinima();
            System.out.println(resultado[0]);
            System.out.println(resultado[1]);

            t2 = System.currentTimeMillis ();

            System.out.println (" i="+i+ "**TIEMPO="+(t2-t1));

        }
    }
}
```

Con este programa, obtuve los siguientes resultados para los diferentes ficheros:

Datos32:

SU DISTANCIA MINIMA: 0,931098

PUNTOS MAS CERCANOS: [27,278446,90,131051] [27,536642,89,236468]

Datos128:

SU DISTANCIA MINIMA: 0,726086

PUNTOS MAS CERCANOS: [91,160139,17,465655] [91,434401,16,793360]

Datos512:

SU DISTANCIA MINIMA: 0,039745

PUNTOS MAS CERCANOS: [68,511846,94,653169] [68,474715,94,638993]

Datos2048:

SU DISTANCIA MINIMA: 0,036184

PUNTOS MAS CERCANOS: [27,692774,55,783120] [27,686908,55,818825]

Datos8192:

SU DISTANCIA MINIMA: 0,009964

PUNTOS MAS CERCANOS: [83,681527,10,289108] [83,691485,10,288760]

Ahora, los tiempos para una matriz generada de forma aleatoria:

n	t(ms)
1024	45
2048	35
4096	25
8192	96
16384	429
32768	fdt

Ahora, el algoritmo de Divide y Vencerás:
A continuación, se muestra la implementación:

```
public static String[]  BuscarDistanciaMinima()
{
    int iz = 0;
    int de = matrizPuntos.length - 1;

    quickSortPorX();

    Object[] resultado = BuscarDistanciaMinimaRec(iz, de);
    double distanciaMinima = (Double) resultado[0];
    int i = (Integer) resultado[1];
    int j = (Integer) resultado[2];
```

```
public static Object[] buscarDistanciaMinimaRec(int iz, int de) {
    double minDistancia = Double.MAX_VALUE;
    int indice1 = -1;
    int indice2 = -1;
    Object[] resultado = new Object[3];

    Object[] resultado1;
    Object[] resultado2;

    Object[] retorno;

    int indiceParticion = (iz + de) / 2;
    int indiceCritico1 = indiceParticion;
    int indiceCritico2 = indiceParticion + 1;

    if (de == iz) {
        resultado[0] = minDistancia;
        resultado[1] = indice1;
        resultado[2] = indice2;
        return resultado;
    }

    if (de - iz == 1) {
        double x1 = matrizPuntos[iz][0];
        double y1 = matrizPuntos[iz][1];

        double x2 = matrizPuntos[de][0];
        double y2 = matrizPuntos[de][1];

        // formula de la distancia entre 2 puntos: Raiz((x2-x1)^2 + (y2-y1)^2)
        double distancia = calcularDistancia(x1, y1, x2, y2);

        if (distancia < minDistancia) {
            minDistancia = distancia;
            indice1 = iz;
            indice2 = de;
```

```java
        if (distancia < minDistancia) {
            minDistancia = distancia;
            indice1 = iz;
            indice2 = de;

            resultado[0] = minDistancia;
            resultado[1] = indice1;
            resultado[2] = indice2;

            return resultado;
        }
    }

    resultado1 = buscarDistanciaMinimaRec(iz, indiceParticion);

    resultado2 = buscarDistanciaMinimaRec(indiceParticion + 1, de);

    retorno = combinar(resultado1, resultado2);

    retorno = comprobarPuntosCriticos(retorno, indiceCritico1, indiceCritico2);

    return retorno;
}

public static double calcularDistancia(double x1, double y1, double x2, double y2) {
    return Math.sqrt((Math.pow((x2 - x1), b: 2)) + Math.pow((y2 - y1), b: 2));
}

public static Object[] combinar(Object[] resultado1, Object[] resultado2) {
    double distancia1 = (Double) resultado1[0];
    double distancia2 = (Double) resultado2[0];

    return distancia1 < distancia2 ? resultado1 : resultado2;
}
```

```java
public static Object[] comprobarPuntosCriticos(Object[] resultadoProvisional, int indiceCritico1, int indiceCritico2) {
    double x1 = matrizPuntos[indiceCritico1][0];
    double y1 = matrizPuntos[indiceCritico1][1];
    double x2 = matrizPuntos[indiceCritico2][0];
    double y2 = matrizPuntos[indiceCritico2][1];

    double distancia = calcularDistancia(x1, y1, x2, y2);
    double distanciaRecurs = (Double) resultadoProvisional[0];
    if (distancia < distanciaRecurs) {
        resultadoProvisional[0] = distancia;
        resultadoProvisional[1] = indiceCritico1;
        resultadoProvisional[2] = indiceCritico2;
    }

    return resultadoProvisional;
}
```

Y después la clase que mide los tiempos:

```java
Run | Debug
public static void main(String[] args)
{
    long t1,t2;
    String[] resultado;

    for (int i = 1024; i <= 10000000; i *= 2)
    {
        PuntosDyV.generarMatrizPuntosAleatoria(i);

        t1 = System.currentTimeMillis ();

        resultado = PuntosDyV.BuscarDistanciaMinima();
        System.out.println(resultado[0]);
        System.out.println(resultado[1]);

        t2 = System.currentTimeMillis ();

        System.out.println (" i="+i+ "**TIEMPO="+(t2-t1));

    }
}
```

La complejidad del algoritmo es:

a = 2
b = 2
k = 1 (O(n * log n) por Quicksort)
 La complejidad es O(n * log n)

A continuación, muestro los resultados para algunos de los ficheros dados:

Datos32:
SU DISTANCIA MINIMA: 0,931098

PUNTOS MAS CERCANOS: [27,278446,90,131051] [27,536642,89,236468]

Datos128:
SU DISTANCIA MINIMA: 0,931098

PUNTOS MAS CERCANOS: [27,278446,90,131051] [27,536642,89,236468]

Datos512:
SU DISTANCIA MINIMA: 0,039745

PUNTOS MAS CERCANOS: [68,474715,94,638993] [68,511846,94,653169]

Datos2048:
SU DISTANCIA MINIMA: 0,039745

PUNTOS MAS CERCANOS: [68,474715,94,638993] [68,511846,94,653169]

Datos8192:
SU DISTANCIA MINIMA: 0,009964

PUNTOS MAS CERCANOS: [83,681527,10,289108] [83,691485,10,288760]

Y ahora los tiempos de ejecución para matrices aleatorias creadas con los siguientes tamaños n:

n	t(ms)
1024	20
2048	1
4096	1
8192	1
16384	3
32768	5
65536	15
131072	32
262144	77
524288	156

Diría que los tiempos de cada algoritmo se corresponden con su complejidad.

Tarea Practica 4 Iker Núñez UO306425

Para empezar, se pedía implementar el algoritmo en Java:

```java
public class ColoreoGrafo {

    public static Map<String, String> realizarVoraz(Map<String, List<String>> grafo){
        String[] colours = {"red", "blue", "green",
        "yellow", "orange", "purple", "cyan", "magenta", "lime"};
        Map<String,String> result = new HashMap<String,String>();

        List<String> vecinos;
        HashSet<String> coloursUsed;

        for(String nodo : grafo.keySet()){
            vecinos = grafo.get(nodo);
            coloursUsed = new HashSet<String>();

            for(String nodoVecino : vecinos){
                if(result.containsKey(nodoVecino)){
                    coloursUsed.add(result.get(nodoVecino));
                }
            }

            for(int j = 0; j < colours.length; j++){
                if(!coloursUsed.contains(colours[j])){
                    result.put(nodo, colours[j]);
                    break;
                }
            }
        }

        return result;
    }
}
```

El heurístico que escogí fue usar el primer color disponible de la lista que no este usado por ninguno de los nodos adyacentes al que se mira.

Tras contrastar los resultados, fueron correctos.

```java
public class DevoradorTiempos {
    Run | Debug
    public static void main(String[] args){
        String[] ficheros = {"g4.json", "g8.json", "g16.json", "g32.json", "g64.json", "g128.json",
            "g256.json","g512.json","g1024.json","g2048.json","g4096.json","g8192.json",
            "g16384.json", "g32768.json","g65536.json"};

        long t1;
        long t2;
        double tiempo;
        for(String fichero : ficheros){
            t1 = System.currentTimeMillis();

            Devorador.main(new String[]{fichero});

            t2 = System.currentTimeMillis();

            tiempo = t2 - t1;

            System.out.printf(format: "Fichero: %s ; Tiempo: %f", fichero, tiempo);
            System.out.println();
        }
    }
}
```

La complejidad de este algoritmo parece cuadrática, ya que hay 2 bucles for anidados. Sin embargo, en el bucle for que recorre los vecinos, nunca se va a recorrer todo el grafo, solo el subgrafo que forman los vecinos del nodo. Por esta razón, la complejidad se aproxima a O(n).

los tiempos que salen, serían correspondientes a una complejidad aproximadamente de O(n):

n	tColoreado(ms)
4	27
8	2
16	3
32	5
64	3
128	4
256	6
512	12
1024	35
2048	23
4096	32
8192	39
16384	89
32768	167
65536	294

Si analizamos el algoritmo, vemos que para cada nodo, se recorren sus vecinos y después el array de colores. Pero claro, los nodos no tienen muchos vecinos, y en cuanto a los colores, para las matrices usadas no se usan todos los colores, luego los fors internos no se ejecutan completamente. Esta podría ser una explicación para que los tiempos obtenidos no se correspondan con la complejidad del algoritmo.

Después, se pide la implementación en Python que sería la siguiente:

```python
def realizar_voraz(grafo):
    colours = ["red", "blue", "green",
        "yellow", "orange", "purple", "cyan", "magenta", "lime"]

    result = {}
    for nodo in grafo:
        vecinos = grafo[nodo]
        coloursUsed = []

        for i in range(len(vecinos)):
            nodoVecino = vecinos[i]

            coloursUsed = {result[str(v)] for v in vecinos if str(v) in result}

        for j in range(len(colours)):
            if(not(colours[j] in coloursUsed)):
                result[nodo] = colours[j]
                break

    return result
```

El heurístico es el mismo que en la implementación de Java y el resultado también es correcto. En cuanto a la complejidad, sucede lo mismo, midiendo los tiempos con esta clase:

```python
def main():
    ficheros = ["g4.json", "g8.json", "g16.json", "g32.json", "g64.json", "g128.json",
            "g256.json","g512.json","g1024.json","g2048.json","g4096.json","g8192.json",
            "g16384.json", "g32768.json","g65536.json"]

    t1 = 0
    t2 = 0
    tiempo = 0

    for fichero in ficheros:
        nombreFichero = "sols/"+fichero
        with open(nombreFichero) as f:
            mapa = json.load(f)
            f.close()

        t1 = time()

        aux.realizar_voraz(mapa["grafo"])

        t2 = time()

        tiempo = (t2 - t1)*1000

        print("Fichero: ", fichero, " tiempo: ",tiempo )
        print("\n")
```

se obtienen los siguientes tiempos:

n	tColoreado(ms)
4	0,0000
8	0,0000
16	0,0000
32	0,0000
64	0,0000
128	0,0000
256	0,0000
512	0,0000
1024	0,0000
2048	16,3910
4096	14,7933
8192	25,9991
16384	37,7975
32768	207,6943
65536	324,4348

Tras analizar los tiempos obtenidos en ambos casos, estos se corresponden con la complejidad, si bien en el caso de Python son menos precisos.

Tarea Práctica 5 Iker Núñez UO306425

1) Definición del estado

En mi caso, utilizo una matriz de dos dimensiones, donde las filas representan el número de vehículos y las columnas representan la ocupación de babor. La ocupación de estribor no hace falta representarla en la matriz, ya que la podemos deducir de la siguiente formula: Ocupación Estribor = SumaAcumuladaLongitudes – Ocupación Babor.

2) Relación de recurrencia

Caso base -> es el estado de la fila 0. La celda (0,0) es siempre **true**, ya que con 0 vehículos cargados tienes una ocupación de cero metros en babor. El resto de las columnas de esta fila se inicializan a **false** ya que no hay vehículos cargados.

Una celda cualquiera de la matriz se pondrá a **true** si cumple una de estas dos condiciones:

-Condición babor:

El vehículo se puede meter en babor. Para esto, debemos estar en una columna que iguale al menos los metros que ocupa el vehículo (no podemos decir que un vehículo de 4 metros ocupa 3 metros en babor). La segunda parte de la condición es que la celda (a,b) sea **true**, donde a es la fila anterior y b es la columna en la que estas menos la longitud del vehículo que metes.

¿Por qué? Porque si dices que ahora la ocupación de babor va a ser la que había mas la longitud del nuevo vehículo, entonces antes la ocupación antes debía ser la de la columna en la que estas menos la long del vehículo (j – longVeh).

La condición de babor en código se vería así:

```
//¿Puedes meter en babor?
if(j >= longVeh && matrizEstado[i - 1][j - longVeh])
```

-Condición estribor:

El vehículo se puede meter en estribor. Esta condición, como la anterior tiene 2 partes. La primera, la celda (i-1,j), es decir, la que esta inmediatamente encima, debe ser **true.** ¿Por qué? Porque si metes un coche en estribor, la ocupación de babor no cambiara, luego miras que babor tenga esos metros ocupados.

La segunda parte de la condición es que el vehículo que quieres meter en estribor

coja en este, para ello miras que la longitud de estribor (incluyendo ya este nuevo vehículo) sea menor o igual que la máxima por carril.

La condición de estribor en código se vería así:

```
//Si no, ¿puedo meter en estribor?
else if(matrizEstado[i-1][j] && longEstribor <= longitudCarril)
```

3) *Complejidad del algoritmo en función de N (nº vehículos) y L (longitud del carril)*

La complejidad de este algoritmo en función de N y L, resulta ser de O(N * L), ya que, para cada vehículo, recorres todas las L columnas.

4) *Implementación*

En la carpeta se adjunta la clase Ferry.java

Tarea Práctica 6 Iker Núñez UO306425

El código resultante para el problema de Bin Packing es el siguiente: Por una parte, la clase que ejecuta el algoritmo:

```
// Solves the problem by backtracking
private void backtracking(int indexS, ArrayList<ArrayList<Integer>> distribution) {
    numberOfRecCalls++;

    // Base case
    if (indexS == setsS.length) {
        Integer thisK = distribution.size();
        if (thisK < bestK) {
            copySolution(distribution);
            return;
        }
    }

    // Pruning: if current size es greater or equal than best one, recursive call
    // ends
    // if (distribution.size() >= bestK) {
    //      // stops the execution
    //      return;
    // }

    for (int pos = 0; pos < distribution.size(); pos++) {
        // tries to add it to an existant container
        if (!setsFlag[indexS] && sum(distribution.get(pos)) + setsS[indexS] <= capacityC) {
            distribution.get(pos).add(setsS[indexS]);
            setsFlag[indexS] = true;
            backtracking(indexS + 1, distribution);
            setsFlag[indexS] = false;
            if (distribution.size() != 0) {
                distribution.get(pos).removeLast();
            }
```

```java
    for (int pos = 0; pos < distribution.size(); pos++) {
        // tries to add it to an existant container
        if (!setsFlag[indexS] && sum(distribution.get(pos)) + setsS[indexS] <= capacityC) {
            distribution.get(pos).add(setsS[indexS]);
            setsFlag[indexS] = true;
            backtracking(indexS + 1, distribution);
            setsFlag[indexS] = false;
            if (distribution.size() != 0) {
                distribution.get(pos).removeLast();
            }
        }
    }

    // could not add it to an existant one, creates a new container
    ArrayList<Integer> newContainer = new ArrayList<Integer>();
    newContainer.add(setsS[indexS]);
    setsFlag[indexS] = true;
    int indexNewInsertion = distribution.size();
    distribution.add(newContainer);
    backtracking(indexS + 1, distribution);
    setsFlag[indexS] = false;
    if (distribution.size() != 0) {
        distribution.get(indexNewInsertion).removeLast();
    }
}
```

Y por otra parte, la clase que mide los tiempos:

```java
Run | Debug
public static void main(String[] args) {
    String nombreFichero;
    long t1;
    long t2;
    double tResultado;

    System.out.println(x: "BACKTRACKING: BIN PACKING");

    System.out.println(x: "MEDICION DE TIEMPOS: ");
    for (int i = 0; i < 10; i++) {
        nombreFichero = "./test0" + i + ".txt";
        String[] argsMain = { nombreFichero };

        t1 = System.nanoTime();
        for(int j = 10; j < 1000; j++){
            AlmacenajeContenedores.main(argsMain);
        }
        t2 = System.nanoTime();
        tResultado = (t2 - t1) / 1_000_000.0;

        System.out.println("FICHERO: " + nombreFichero + ", TIEMPO(ms) = " + tResultado);
    }
    System.out.println(x: "MEDICION DE TIEMPOS FINALIZADA");
}
```

Tabla de medición de tiempos:

fichero	S	t(ms)
test00	10	33,6597
test01	12	3,2356
test02	10	1481,4852
test03	12	34,2125
test04	15	fdt
test05	20	fdt
test06	14	fdt
test07	15	fdt
test08	20	fdt
test09	20	fdt

A pesar de que los tiempos salen un poco extraños, para el test04.txt se tardó +5 min en ejecutar. Esto es porque en los algoritmos de backtracking, la complejidad es exponencial.

En futuras prácticas, veremos algunas técnicas como poda u ordenar el array de entrada y con estas podremos disminuir la complejidad en gran parte.

Tarea Práctica 6-Extra Iker Núñez UO306425

Se pedía codificar un algoritmo que resolviera el problema de las nReinas. El código resultante fue el siguiente:

Por una parte, el código del algoritmo:

```java
public class NReinas {

    private List<List<String>> soluciones; // cada lista interna de Strings representa un tablero completo donde cada Strir

    private char[][] tablero;
    private boolean[] columnas; // indica si una columna específica (de 0 a N-1) ya tiene una reina.
    private boolean[] diagonalesPrincipales; // de arriba a abajo, izda a decha (\)
    private boolean[] diagonalesSecundarias; // de arriba a abajo, decha a izda (/)

    /**
     * Método principal que inicia el algoritmo.
     * @param n de tipo int. El tamaño del tablero y el número de reinas.
     * @return soluciones, de tipo List<List<String>>. Una lista con todas las configuraciones válidas del tablero
     */
    public List<List<String>> resolverNReinas(int n) {
        soluciones = new ArrayList<>();

        // Inicializamos los rastreadores de ataques en columnas y diagonales
        columnas = new boolean[n];
        diagonalesPrincipales = new boolean[2*n-1];
        diagonalesSecundarias = new boolean[2*n-1];

        // Inicializar el tablero vacío con puntos '.' (os doy un método)
        tablero = crearTablero(n);
        // Iniciar el backtracking
        backtracking(0, n, tablero);

        return soluciones;
    }
```

```java
 * Método de backtracking para colocar las reinas en el tablero.
 * @param fila, de tipo int. La fila actual en la que intentamos colocar una reina.
 * @param n,    de tipo int. El tamaño del tablero y el número de reinas.
 * @param tablero,  de tipo char[][]. La representación actual del tablero con las reinas colocadas.
 */
private void backtracking(int fila, int n, char[][] tablero) {
    // Caso base: si logramos llegar a la fila n, colocamos todas las reinas con éxito
    if(fila == n){
        List<String> solucion = construirSolucion(tablero);
        soluciones.add(solucion);
        return;
    }

    for (int col = 0; col < n; col++) {

        // Fórmulas matemáticas para identificar la diagonal a la que pertenece una celda
        int d1 = fila - col + (n - 1); // Diagonal principal (\)
        int d2 = fila + col;           // Diagonal secundaria (/)

        // 1. Avanzar -> Colocar la reina (meter un 'Q') y marcar la columna y diagonales como atacadas
        if(columnas[col] || diagonalesPrincipales[d1] || diagonalesSecundarias[d2]){
            continue;
        }

        columnas[col] = true;
        diagonalesPrincipales[d1] = true;
        diagonalesSecundarias[d2] = true;
        tablero[fila][col] = 'Q';
```

```java
for (int col = 0; col < n; col++) {

    // Fórmulas matemáticas para identificar la diagonal a la que pertenece una celda
    int d1 = fila - col + (n - 1); // Diagonal principal (\)
    int d2 = fila + col;           // Diagonal secundaria (/)

    // 1. Avanzar -> Colocar la reina (meter un 'Q') y marcar la columna y diagonales como atacadas
    if(columnas[col] || diagonalesPrincipales[d1] || diagonalesSecundarias[d2]){
        continue;
    }

    columnas[col] = true;
    diagonalesPrincipales[d1] = true;
    diagonalesSecundarias[d2] = true;
    tablero[fila][col] = 'Q';

    // 2. backtracking
    backtracking(fila + 1, n, tablero);

    // 3. Retroceder
    columnas[col] = false;
    diagonalesPrincipales[d1] = false;
    diagonalesSecundarias[d2] = false;
    tablero[fila][col] = '.';
}
```

Y por otra parte, una clase que mide los tiempos de ejecución:

```java
public class NReinasTiempos{
    private static NReinas nReinas = new NReinas();

    Run | Debug
    public static void main(String[] args){
        long t1;
        long t2;
        double tFinal;

        System.out.println(x: "MEDICION DE TIEMPOS N REINAS: ");

        for(int i = 2; i < 20; i += 2){
            t1 = System.nanoTime();

            nReinas.resolverNReinas(i);

            t2 = System.nanoTime();

            tFinal = (t2 - t1)/ 1_000_000;

            System.out.printf(format: "PARA N = %d , T = %f MS", i, tFinal);
            System.out.println();
        }

        System.out.println(x: "FIN DE LA MEDICION DE TIEMPOS");
    }
}
```

El resultado de la medición de tiempos fue la siguiente:

n	t(ms)
2	0
4	0
6	0
8	2
10	5
12	127
14	4013
16	OutOfMemError

Tarea Práctica 7 Iker Núñez UO306425

En esta práctica, se pretendía mejorar los tiempos del algoritmo hecho en la práctica p6 con backtracking, introduciendo ramificación y poda. Esta técnica, busca evaluar muchos menos casos que en backtracking, usando algunos recursos como la poda inicial, una cota inferior y los heurísticos de ramificación.

En mi caso, la primera mejora que aplique, fue ordenar el array de entrada. Con esta simple modificación, los tiempos ya mejoran notablemente.

Después, ya sobre las técnicas nombradas, las implementé de la siguiente forma:

-Poda inicial:

Se calculó un valor inicial. Este se calculó siguiendo el siguiente heurístico de ramificación: Se coge el array de entrada, y se van cogiendo por pares los elementos de este. Sumas cada par y lo divides entre la capacidad. Los resultados de estas divisiones los debes ir guardando en una variable.

Si el array de entrada fuera impar, debemos hacer la división del elemento restante entre la capacidad y sumarlo a la variable.

En código, se vería así:

```
//Gets the initial "bestK"
private int getBestK(){
    int pairSum = 0;
    int bestK = 0;

    if(setsS.length % 2 == 0){
        for(int i = 0; i < setsS.length - 1; i += 2){
            pairSum = setsS[i] + setsS[i+1];
            bestK += (pairSum % capacityC == 0)? pairSum / capacityC : (pairSum / capacityC) + 1;
        }

        return bestK;
    } else{
        for(int i = 0; i < setsS.length - 2; i += 2){
            pairSum = setsS[i] + setsS[i+1];
            bestK += (pairSum % capacityC == 0)? pairSum / capacityC : (pairSum / capacityC) + 1;
        }

        // add the last one
        int last = setsS[setsS.length - 1];
        bestK += (last % capacityC == 0)? last / capacityC :
            (last / capacityC) + 1;

        return bestK;
    }
}
```

-Cota inferior

Tras calcular la poda inicial, buscamos la posibilidad de descartar más caminos usando una cota inferior. Esta, la calculamos de la siguiente manera:

Pasamos un valor como parámetro que es la suma restante (es la suma de todos los elementos que no se han almacenado aún), y a este valor le sumamos la capacidad, le restamos uno para que no haya problemas de truncamiento y dividimos entre la capacidad. De esta manera, obtendríamos

un valor precalculado del número “supuesto” de contenedores que usaremos para almacenar todos los elementos restantes.

En código se vería así:

```
// Lower bound
// Calculate the minimum theoric number of containers for the space remaining
int lowerBound = (sumaRestante + capacityC - 1)/capacityC;
```

Por último, usando las 2 técnicas anteriores, podemos construir la condición de poda del algoritmo. En código se vería así:

```
  // if the number of containers + lowerBound is greater than bestK, stops this branch execution
  if(distribution.size() + lowerBound > bestK) return;
```

La idea es: si el número de contenedores hasta ese nivel/iteración más el mínimo teórico (lowerBound) es mayor que la mejor solución (si no se encontró solución sería la poda inicial calculada, y si no sería la mejor solución encontrada), entonces descartas ese camino.

De esta manera, dejamos de evaluar muchos caminos que no nos van a llevar a una solución mejor que la ya obtenida.

El resto de código de la clase sería este:

```
public class AlmacenajeContenedoresRyP {
    private int capacityC;
    private Integer[] setsS;
    private boolean[] setsFlag;
    private int bestK; // minimum number of containers

    private int numberOfRecCalls = 0;

    private ArrayList<ArrayList<Integer>> bestDistribution;

    // Constructor
    public AlmacenajeContenedoresRyP(int capacity, Integer[] setsS) {
        this.capacityC = capacity;
        this.setsS = setsS;
        Arrays.sort(this.setsS, Collections.reverseOrder());
        this.setsFlag = new boolean[setsS.length];

        this.bestDistribution = new ArrayList<ArrayList<Integer>>();
        bestK = getBestK(); // Initializes to the worst case
    }
```

```java
    // Solves the problem and logs the solution
    public void searchSolution() {
        // calls the solving algorithm
        backtracking(indexS: 0, new ArrayList<ArrayList<Integer>>(), totalSumS());

        // logs the solution
        logSolution();
    }

    // Solves the problem by backtracking
    private void backtracking(int indexS, ArrayList<ArrayList<Integer>> distribution, int sumaRestante) {
        numberOfRecCalls++;

        // Lower bound
        // Calculate the minimum theoric number of containers for the space remaining
        int lowerBound = (sumaRestante + capacityC - 1)/capacityC;

        // if the number of containers + lowerBound is greater than bestK, stops this branch execution
        if(distribution.size() + lowerBound > bestK) return;

        // Base case
        if (indexS == setsS.length) {
            Integer thisK = distribution.size();
            if (thisK < bestK) {
                bestK = thisK; // updates the bestK
                copySolution(distribution);
            }
            return;
        }
```

```java
    // could not add it to an existant one, creates a new container
    ArrayList<Integer> newContainer = new ArrayList<Integer>();
    newContainer.add(setsS[indexS]);
    setsFlag[indexS] = true;
    distribution.add(newContainer);

    backtracking(indexS + 1, distribution, sumaRestante - setsS[indexS]);

    setsFlag[indexS] = false;
    if (distribution.size() != 0) {
        distribution.remove(distribution.size() - 1);
    }
}

// Returns the sum of the elements in the container
private int sum(ArrayList<Integer> container) {
    Integer sumContainer = 0;

    for (Integer number : container) {
        sumContainer += number;
    }

    return sumContainer;
}
```

```java
    private int totalSumS(){
        Integer totalSum = 0;
        for(Integer number : setsS){
            totalSum += number;
        }

        return totalSum;
    }

    // Copies the solution given by param to save it
    private void copySolution(ArrayList<ArrayList<Integer>> distribution) {
        bestDistribution.clear();

        for (ArrayList<Integer> container : distribution) {
            bestDistribution.add(new ArrayList(container));
        }

        bestK = distribution.size();
    }
```

En tiempos, hay una reducción muy notable, tan notable como que en la p6 algunos casos de test no terminaban, y en este caso sí. A continuación, se muestra una comparación:

Tabla de tiempos p6 (backtracking original):

fichero	S	t(ms)
test00	10	33,6597
test01	12	3,2356
test02	10	1481,4852
test03	12	34,2125
test04	15	fdt
test05	20	fdt
test06	14	fdt
test07	15	fdt
test08	20	fdt
test09	20	fdt

Tabla de tiempos p7 (backtracking + RyP):

fichero	S	t(ms)
test00	10	42,6861
test01	12	3,7473
test02	10	2,5202
test03	12	2,1802
test04	15	3,2794
test05	20	1,5210
test06	15	2,2566
test07	15	2,4747
test08	20	2,2536
test09	20	1,7101

Como se puede observar, la mejora en los tiempos es muy visible. Por lo tanto, podemos decir que nuestras técnicas de poda funcionan a la perfección.